

NOTE

The registration system we've set up allows anyone to make any number of accounts for Learning Log. Some systems require users to confirm their identity by sending a confirmation email that users must reply to. By doing so, the system generates fewer spam accounts than the simple system we're using here. However, when you're learning to build apps, it's perfectly appropriate to practice with a simple user registration system like the one we're using.

TRY IT YOURSELF

19-2. Blog Accounts: Add a user authentication and registration system to the Blog project you started in Exercise 19-1 (page 415). Make sure logged-in users see their username somewhere on the screen and unregistered users see a link to the registration page.

Allowing Users to Own Their Data

Users should be able to enter private data in their learning logs, so we'll create a system to figure out which data belongs to which user. Then we'll restrict access to certain pages so users can only work with their own data.

We'll modify the Topic model so every topic belongs to a specific user. This will also take care of entries, because every entry belongs to a specific topic. We'll start by restricting access to certain pages.

Restricting Access with `@login_required`

Django makes it easy to restrict access to certain pages through the `@login_required` decorator. Recall from Chapter 11 that a *decorator* is a directive placed just before a function definition, which modifies how the function behaves. Let's look at an example.

Restricting Access to the Topics Page

Each topic will be owned by a user, so only registered users can request the topics page. Add the following code to `learning_logs/views.py`:

```
learning_logs/
views.py
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required

from .models import Topic, Entry
--snip--

@login_required
def topics(request):
    """Show all topics."""
    --snip--
```